

COMP101

Introduction to Programming

2025-26

Lecture-05
Numeric Input/Output (I/O)
Casting

Data types in Python

- str ('string data type')
 - single letters, words, symbols, numbers
 - Some languages (not Python) treat a single letter as a character ('char data type')
- int ('numeric data type integer')
 - integer numbers have no decimal places
- float ('numeric data type real')
 - real numbers have decimal places
- bool ('Boolean data type')
 - True or False, 1 or 0, Yes or No

Numeric data types

Assigning ('allocating') to a variable ('var')

- To use numeric data types integer or float (real) we construct a 'variable declaration' – we 'assign them to a variable' (var)
- Assignment can be done 'explicitly' or 'implicitly' using a set of rules
 - We think up a name for the variable, place a '=' sign after it and then think of the data that is to be assigned to the variable

Just the same as when
inputting strings

→ **user-defined variable name = data_value**

The variable name can be any name that is not a 'reserved word' (words that Python uses for special purposes).
Some other rules apply, e.g. it can't start with a number or a symbol etc

This is the 'assignment operator'
It isn't the 'equals sign' although we usually end up saying this way

This can be a str, int, float or bool

Explicit Assignment - numeric data types

- User does not input anything – values are assigned by the programmer within the code

`firstNumber = 21`

`#var firstNumber is assigned integer value 21`

`secondNumber = 3.142`

`#var secondNumber is assigned float value 3.142`

If you intend to use pi, why not call it pi

`pi = 3.142`

`#var name more accurately reflects the float value – easier for user`

Explicit Assignment – numeric data types

`firstNumber = 21` `#explicitly assigned integer var`

`pi = 3.142` `#explicitly assigned float var`

If we change the value of var pi within the code as follows:

`pi = 3.1`

Value 3.1 will be the value of pi from this point on – the original value of 3.142 is overwritten and is lost

If we wanted to use both 3.142 and 3.1 then we must use a different var name for each value of pi

`pi = 3.142`

`piShort = 3.1`

Explicit assignment is useful when we want to declare a 'constant';
Use CAPITALS to remind you not to change it in the code

Explicit Assignment

Dynamic Allocation of data type

Python 'dynamically allocates' the var to the assigned data type

`firstNumber = 21` #Python 'knows' 21 is integer - firstNumber is an 'integer var'

`pi = 3.142` #Python 'knows' 3.142 is float – pi is a 'float var'

Python reads the data to the right of the assignment operator and dynamically allocates it to the variable name. The var name now holds a value and the data type of that value.

Other languages may use 'static allocation' –which must declare the data type before use:

e.g. at the top of the program, the programmer has to type:

`firstNumber: int;`

`pi: float;`

Further down the coding when we code `firstNumber = 21` the language is expecting an integer input and will 'throw an error' if it is not integer

Implicit Allocation – numeric data types

- When we execute a program we usually want it to run with different values for each run. This can't be done with explicit allocation
- When we want user to input a value each time the program runs – it needs a change to the statement

`user-defined variable name = input ("prompt user for input")`

`firstNumber = input ("Enter a number: ")` 21

`pi = input ("Enter a value of pi to 3 dp: ")` 3.142

Execution of the program stops until user inputs a number and presses return. Each time the program is run, user can enter different values

I/O Numbers

- **PROBLEM:** `input()` function defaults to string
- Enter a number - we can't do maths on it (it's a string data type)

```
age_now = input ("Enter your age in years: ") 21      #this is string '21' – the default type for input()  
age_now = (age_now + 1)  
print (age_now)
```

Traceback (most recent call last):

File "C:/Users/dures/Documents/age.py", line 2, in <module>

```
age = age+1
```

TypeError: can only concatenate str (not "int") to str

We can't do maths on strings so we get a 'TypeError':
TypeError means trying to code an operation (e.g. addition) on a type that is not supported by the operation;
i.e. you can't add numbers and strings together

I/O Numbers

Casting ('Data type conversion')

- Casting fixes the problem of input() defaulting to string
 - It converts the input data type to a different data type
 - Add an extra instruction to the input() function

```
age_now = input("Enter a number: ") 21
```

#this is string '21' – can't do maths

#use a cast ('int') to change the data type from string to integer

```
age_now = int(input("Enter a number: ")) 21
```

#this is integer 21 – can do maths

```
age_now = age_now+1
```

```
print (age_now)
```

22

Note the syntax – where the words 'int' and 'input' go and the nested parentheses

If you cast to integer and then input a float you get:

ValueError: invalid literal for int() with base 10: "

I/O Numbers – Casting to float

- Cast str to float (because we want decimal places here)

#use cast 'float'

```
data_entry = float(input("Enter a number: ")) 21
```

```
print (data_entry)
```

21.0

#now it's a real number (float) 21.0 – can do maths

#note the decimal point and place is added automatically

Use extra parentheses and the word 'float'.
String '21' is converted to float 21.0

You can input float or an integer which
converts to a float on output

I/O Numbers - Casting to string

- Cast float to str

#use 'str'

```
data_entry = str(input("Enter a real number: ")) 21.0
```

```
print (data_entry)
```

```
21.0
```

Use extra parentheses and the word 'str'.
Float 21.0 is converted to string '21.0'

- Python does not have a const declaration. You could use str() to keep a number from being altered in the code later on – but it can cause an error if the user tries to do maths on it

I/O Numbers - Casting Summary

- `data_entry = 21` #var is integer
- `data_entry = 3.142` #var is float
- `data_entry = input("Enter a number: ")` 21 #var is string
- `data_entry = int(input("Enter a number: "))` 21 #var is integer
- `data_entry = float(input("Enter a number: "))` 21 #var is float
- `data_entry = str(data_entry)` 21 #var is string

In the examples above, we have made a 'declaration'.

The first two examples show 'explicit declaration' – there is no user input

The last four examples show 'implicit declaration' – it needs user input

var Overwriting

- 'Overwriting' – putting a new value into the original var
- This happens when we use the same var name

#var age_now has cast string input '21' to integer 21

```
age_now = int(input("Enter your age in years: ")) 21
```

```
print(age_now)
```

21

#add 10 to age_now using the same var name

```
age_now = (age_now + 10) #parentheses aid readability
```

```
print(age_now)
```

31

We have 'lost' the original value (21) of var 'age_now' – it has been over-written with the value 31

Var Overwriting

- Keep the original value after changing it - use a new var name

#cast string '21' to integer 21

```
age_now = int(input("Enter your age in years: ")) 21
```

```
print(age_now)
```

```
21
```

#new var age_in_ten holds the new value and we keep the original in age_now

```
age_in_ten = (age_now + 10)
```

```
print(age_in_ten)
```

```
31
```

Check which var you want to use as we now have two vars in use:
age_in_ten has value 31
age_now still has value 21

Var Overwriting

#cast string to float: string '21' casts to float 21.0

```
age_now = float(input("Enter your age in years: ")) 21  
print(age_now)  
21.0
```

#new variable declared to hold the new value else we'll lose value 21.0

```
age_in_ten = (age_now + 10)  
print(age_in_ten)  
31.0
```

We are adding integer value of 10 to a float value of 21.0
Python defaults the answer to a float

Output multiple variables (best not to do it)

```
stud_name = input("Enter your name: ") John
```

```
stud_age = input ("Enter your age: ") 21      #this is string '21' – no maths?
```

```
stud_year = int(input ("Enter your start year on the course: ")) 2025
```

#be careful with this form of output

```
print (stud_name,stud_age,stud_year)
```

John 21 2022

The commas in the output string put spaces between the vars on output

Outputting multiple vars may not be the best way to handle output.
Can lead to errors and confusion, usually in the ordering of the output.
The order of the vars in the parentheses is the order in which they will output